



TREBALL FINAL DE GRAU



ESCOLA
POLITÈCNICA SUPERIOR
UNIVERSITAT DE LLEIDA
INSPIRING THE FUTURE

Estudiant: **Xiao Long Ji**

Titulació: Grau en Enginyeria Informàtica

Títol de Treball Final de Grau: **Comparativa de Systemd amb Kubernetes a l'enton Edge**

Director/a: **Oriol Agost Batalla, Jordi Mateo Fornés**

Presentació

Mes: Setembre

Any: 2026

Acknowledgements

Vull expressar el meu sincer agraïment als meus tutors pel seu guiatge inestimable, paciència i dedicació al llarg de tot el desenvolupament d'aquest projecte. La seva expertesa i els seus consells constants han estat fonamentals per donar forma a aquesta recerca.

També voldria donar les gràcies a la meua família pel seu suport incondicional i acompanyament en tot moment.

Abstract

En l'Edge Computing, els dispositius perimetrals tenen recursos extremadament limitats. El desplegament d'orquestradors moderns de contenidors genera una reserva permanent de memòria i CPU que redueix significativament el marge disponible per processar càrregues de treball exigents, com ara la Intel·ligència Artificial.

L'objectiu principal és avaluar el sobrecost de rendiment que introdueix Kubernetes en maquinari modest en comparació amb una solució d'execució directa basada en Systemd, per tal d'entendre quin sistema compensa més.

En quant a la metodologia, s'han executat proves d'estrès automatitzades per a ambdues arquitectures utilitzant els diferents protocols de comunicació com http-json, grpc-protobuf i ZeroMQ. L'avaluació s'ha realitzat mesurant el consum de RAM en repòs, la degradació del rendiment de xarxa (Throughput), i la capacitat d'auto-recuperació aplicant l'eliminació de processos (kill-9) per calcular el Temps Mitjà de Recuperació (MTTR).

Els resultats demostren que l'adopció de Kubernetes penalitza significativament els recursos disponibles al node perimetral. L'orquestrador incrementa l'ús base de memòria en un 16% (aproximadament 850 MB enfront dels 730 MB de Systemd) per la necessitat de mantenir actius els seus processos de control, i la seva capa de xarxa virtual (SDN) redueix el rendiment de processament en un 30% (caient a unes 6.500 imatges per segon enfront de les 9.300 assolides nativament). En termes d'auto-recuperació, la gestió a nivell de sistema operatiu de Systemd assoleix reiniciar els serveis caiguts tres vegades més ràpid (490 ms) que el model distribuït de Kubernetes (1600 ms).

Systemd és la solució idònia per a nodes Edge aïllats on el rendiment brut i la latència són crítics. El consum addicional de recursos que imposa Kubernetes només es justifica en entorns on prima la reducció de la complexitat de gestió, com ara flotes distribuïdes que requereixen actualitzacions declaratives i orquestració centralitzada.

Índex

Acknowledgements	i
Abstract	iii
1 Introduction	1
1.1 Motivació	1
1.2 Objectius	1
1.3 Abast	2
1.4 Estructura del document	2
2 Background	3
2.1 Tecnologies de Virtualització i Contenedors	3
2.2 L'Evolució de l'Orquestració i la Falsa Dicotomia a l' <i>Edge</i>	3
2.3 L'Enfocament Natiu: El paradigma de Systemd, Quadlets i Ansible	4
2.4 Transmissió de Dades i Protocols de Comunicació	4
2.5 Càrregues de treball exigents: La necessitat d'optimitzar per a la IA	5
3 State of the Art	7
3.1 Solucions d'Orquestració Lleugera a l' <i>Edge</i> (K3s, MicroK8s)	7
3.2 Execució Nativa de Contenedors (<i>Standalone</i>)	7
3.3 Avaluació de Protocols de Comunicació per a IA	7
3.4 El buit que omple aquest treball	7
4 Methodology	9
4.1 Disseny Iteratiu i Selecció de Protocols	9
4.2 Selecció del Model d'IA	10
4.3 Topologia i Estratègia de Desplegament (Patró <i>Air-Gapped</i>)	10
4.4 Automatització i Reproduïbilitat (<i>Pipeline</i> d'execució)	10
4.5 Tècniques de mesura i Mètriques d'Avaluació	11
4.5.1 Tractament de Dades (Data Cleansing)	11
5 Implementation	13
5.1 Tecnologies, Llenguatges i Eines Utilitzades	13
5.2 Disseny i Entrenament del Model d'IA (PyTorch)	13
5.3 Infraestructura com a Codi: Arquitectura Nativa (Systemd)	15
5.4 Infraestructura com a Codi: Arquitectura Distribuïda (K3s)	16
5.5 Disseny del Protocol de Comunicació i Inferència d'IA	17
5.6 Automatització del <i>Pipeline</i> i <i>Data Lake</i>	18
5.7 Decisions Tècniques i Resolució de Problemes	20

6	Results	23
6.1	Selecció del Protocol de Comunicació	23
6.2	Rendiment del Model d'IA (CNN)	25
6.3	Comparativa Arquitectònica a l'Edge: Systemd vs Kubernetes (K3s)	27
6.3.1	L'Impost Arquitectònic (Consum de RAM en Repòs)	28
6.3.2	Enginyeria del Caos i Resiliència (MTTR)	28
6.3.3	Penalització de Xarxa Virtual (Rendiment i Throughput)	29
6.4	Conclusions	30
7	Conclusions and Future Work	31
7.1	Assoliment d'Objectius	31
7.2	Contribucions Principals	31
7.3	Limitacions Conegudes	31
7.4	Línies de Treball Futur	32

Índex de figures

2.1	Datasets MNIST amb imatges de dígit manuscrits classificats.	5
2.2	Arquitectura comú de xarxes MLP y CNN dissenyades per a classificació y regresió. Font: Adaptada de [1].	6
5.1	Arquitectura de l'entorn nativa amb Systemd.	15
5.2	Arquitectura de l'entorn nativa amb Kubernetes.	16
5.3	Topologia completa de la inferència d'IA sobre Systemd.	17
5.4	Topologia completa de la inferència d'IA sobre K3s.	18
5.5	Flux del pipeline d'execució automatitzada.	19
6.1	Comparativa de rendiment brut (<i>Throughput</i>) processant tensors.	24
6.2	Impacte de la serialització en el consum màxim de memòria RAM als <i>Workers</i>	24
6.3	Relació de compromís (<i>Trade-off</i>) entre rendiment global i Latència RTT.	25
6.4	Avaluació del model fundacional: Corba de pèrdua (<i>Train Loss</i>).	26
6.5	Avaluació del model fundacional: Matriu de confusió en validació.	26
6.6	Exemples d'inferència del model fundacional: predicció i valor real al node Worker.	27
6.7	Comparativa de la petjada de memòria base (<i>Idle RAM Footprint</i>) als <i>Workers</i>	28
6.8	Temps Mitjà de Recuperació (MTTR) davant la destrucció abrupta d'un contenidor.	29
6.9	Penalització del rendiment màxim d'injecció a causa del CNI de Kubernetes.	29

Índex de taules

6.1	Resultats consolidats de rendiment de xarxa per protocol a l'Edge.	23
6.2	Comparativa de rendiment: Protocol 04-zeromq-messagepack (Basat en 6 tests històrics)	27

Capítol 1

Introduction

Aquest Treball de Fi de Grau (TFG) presenta una avaluació comparativa entre dues arquitectures d'execució de contenidors en entorns Edge: una solució d'execució nativa i directa basada en Systemd i un orquestrador modern com Kubernetes (en la seva versió lleugera K3s).

1.1 Motivació

Als entorns perifèrics, els dispositius operen amb un maquinari molt limitat. Aquesta realitat xoca amb el desplegament d'orquestradors moderns com Kubernetes, els quals exigeixen mantenir actius múltiples processos en segon pla (com el kubelet o els controladors de xarxa) només per funcionar. Aquest consum permanent actua com un llast de memòria i CPU que retalla dràsticament el marge disponible per processar tasques exigents, com ara la Intel·ligència Artificial. Per això, resulta rellevant investigar alternatives d'execució directa a nivell de sistema operatiu per esbrinar si es pot recuperar aquesta potència perduda sense sacrificar la fiabilitat de l'aplicació.

1.2 Objectius

L'objectiu principal és avaluar el sobrecost de rendiment que introdueix Kubernetes (K3s) en maquinari modest en executar càrregues d'Intel·ligència Artificial (específicament, l'entrenament d'un model MNIST amb PyTorch), comparant-lo amb una solució d'execució directa orquestrada mitjançant la combinació de Systemd, Ansible i Podman.

Per assolir-ho, es plantegen els següents objectius específics:

- Mesurar l'impacte de la infraestructura avaluant mètriques base com el temps de desplegament (T_{deploy}), la CPU en repòs i la memòria RAM en repòs a nivell de sistema operatiu.
- Avaluar el rendiment de xarxa sotmetent el sistema a càrrega d'estrès amb quatre protocols de comunicació diferents: HTTP-JSON, gRPC-Protobuf, ZeroMQ-Protobuf i ZeroMQ-MessagePack.
- Analitzar l'eficiència i la penalització de la xarxa virtual recopilant mètriques detallades de comunicació, incloent-hi el Temps Total, el *Throughput*, la Taxa d'Èxit (%), la latència RTT, el Temps de Processament (T_{Proc}), els pics màxims de RAM i CPU, i la mida del Payload en MB.
- Determinar la velocitat de recuperació davant d'errors (MTTR) eliminant contenidors de manera abrupta per forçar la reacció del sistema i comparar la resiliència nativa enfront de la de l'orquestrador.

- Automatitzar els desplegaments i l'extracció de totes aquestes dades mitjançant *scripts* per generar un conjunt de resultats estructurat i assegurar que l'experiment sigui totalment reproducible.

1.3 Abast

L'estudi es delimita a l'avaluació nodes Edge aïllats, comparant la versió lleugera de Kubernetes (K3s) amb la integració de Podman i Systemd mitjançant Quadlets executant càrregues d'entrenament d'un model d'intel·ligència artificial basat en el conjunt de dades MNIST amb PyTorch a cada node worker. S'inclou l'execució de proves d'estrès automatitzades sobre diversos protocols (HTTP-JSON, gRPC-Protobuf, ZeroMQ-Protobuf i ZeroMQ-MessagePack).

L'avaluació quantitativa es divideix en dues categories principals extractes automàticament: d'una banda, les mètriques d'infraestructura (systemd i kubernetes) per calcular l'impacte base, incloent-hi el temps de desplegament (T_deploy), el consum de CPU i RAM en repòs, i el temps mitjà de recuperació (MTTR); de l'altra, les mètriques detallades de xarxa (HTTP-JSON, gRPC-Protobuf, ZeroMQ-Protobuf i ZeroMQ-MessagePack) per analitzar l'eficiència de la comunicació sota càrrega, recollint el temps total, la capacitat de processament (Throughput), la taxa d'èxit (%), la latència (RTT), el temps de processament (T_Proc), els pics màxims de recursos (RAM_Max, CPU_Max) i la mida de la càrrega útil transferida (Payload_MB).

Queden excloses d'aquest treball les distribucions tradicionals de Kubernetes de gran format orientades al núvol (Cloud), l'estudi d'arquitectures d'alta disponibilitat (HA) per al node de control i l'avaluació d'altres motors de contenidors estructurats amb dimoni, com ara Docker.

1.4 Estructura del document

Aquest document s'estructura en set capítols principals que descriuen el disseny, desenvolupament i avaluació empírica del projecte:

Capítol 1 (Introducció): Exposar la motivació, els objectius principals i l'abast del projecte, presentant la problemàtica del sobrecost arquitectònic en entorns perimetral.

Capítol 2 (Marc Teòric): Defineix els conceptes tècnics fonamentals, incloent-hi els motors de contenidors, els protocols de xarxa avaluats, i l'arquitectura de solucions com Systemd, Ansible i Kubernetes.

Capítol 3 (Estat de l'Art): Revisa el context tecnològic actual i les aproximacions existents a la indústria per gestionar càrregues de treball en dispositius *Edge* amb recursos limitats.

Capítol 4 (Metodologia): Detalla el disseny de l'experiment, descrivint l'entorn de proves, les eines utilitzades i definint amb precisió les mètriques d'infraestructura i comunicació a monitorar.

Capítol 5 (Implementació): Explica en profunditat el desenvolupament de la solució, des de la creació dels contenidors i la integració nativa amb Quadlets, fins a l'automatització total dels desplegaments i l'extracció de dades.

Capítol 6 (Resultats): Presenta i analitza les dades obtingudes, contrastant de manera directa el consum de memòria, la capacitat de processament de xarxa (*Throughput*) i la resiliència (MTTR) d'ambdues arquitectures.

Capítol 7 (Conclusions): Avalua l'acompliment dels objectius marcats, consolida el veredict sobre els compromisos tècnics (trade-offs) i proposa línies futures de treball.

Capítol 2

Background

Aquest apartat estableix la base teòrica necessària per comprendre l'evolució dels contenidors i l'impacte arquitectònic que diferents models d'orquestració tenen sobre l'execució de càrregues d'Intel·ligència Artificial en entorns amb recursos limitats.

2.1 Tecnologies de Virtualització i Contenidors

Els contenidors són una tecnologia de virtualització a nivell de sistema operatiu que permet empaquetar una aplicació i totes les seves dependències en una unitat estandarditzada. A diferència de les màquines virtuals, els contenidors comparteixen el nucli (*kernel*) del sistema amfitrió, la qual cosa els fa molt més lleugers, ràpids d'iniciar i eficients en l'ús de CPU i memòria.

Arquitectures amb dimoni i sense dimoni (*Daemonless*): Tradicionalment, motors com Docker funcionen basant-se en un dimoni (*daemon*), un procés central en segon pla (generalment executat amb privilegis d'administrador) que gestiona tot el cicle de vida dels contenidors. Si aquest dimoni cau, tot el sistema s'atura. Com a evolució, sorgeix Podman, un motor de contenidors sense dimoni. Podman executa els contenidors interactuant directament amb el nucli del sistema mitjançant processos independents [2], fet que millora la seguretat, redueix el consum en repòs i permet una integració molt més natural amb el sistema operatiu de la màquina.

2.2 L'Evolució de l'Orquestració i la Falsa Dicotomia a l'Edge

L'orquestració automatitza el desplegament, l'escalabilitat i la resiliència de múltiples contenidors. Històricament, l'evolució d'aquests sistemes i el seu mapa mental a la indústria ha travessat diferents fases:

1. **La Guerra dels Orquestradors i el domini del *Cloud*:** En els inicis de la contenerització, diverses tecnologies (com Docker Swarm o Apache Mesos) van competir per liderar l'orquestració. Cap a l'any 2018, Kubernetes va guanyar aquesta batalla tecnològica de forma absoluta, convertint-se en l'estàndard indiscutible per als grans centres de dades (*Cloud*).
2. **El dogma de "Kubernetes cada vegada més lleuger":** Quan va sorgir la necessitat d'executar càrregues de treball als extrems de la xarxa (*Edge Computing*), la indústria va assumir una visió de túnel: l'única forma de fer-ho era "encongir" Kubernetes. Els esforços d'enginyeria es van centrar exclusivament a retallar components massius per encabir l'orquestrador en dispositius petits, donant lloc a distribucions com K3s o MicroK8s. Tanmateix, totes elles mantien obligatòriament l'arquitectura base: un procés resident (*kubelet*) en cada node i xarxes virtuals internes (SDN).
3. **El buit científic:** Aquesta tendència va consolidar una falsa dicotomia tècnica en la literatura:

- S'havia d'acceptar la penalització permanent de memòria i CPU d'un Kubernetes lleuger per mantenir el clúster viu.
- O bé s'havien d'utilitzar contenidors autònoms (*standalone*) de manera aïllada, sacrificant per complet la gestió centralitzada i l'auto-recuperació davant de fallades.

2.3 L'Enfocament Natiu: El paradigma de Systemd, Quadlets i Ansible

Per trencar aquesta dicotomia, neix una iniciativa tècnica impulsada per Red Hat que planteja un canvi de paradigma fonamental: en lloc d'introduir un agent orquestrador extern (kubernetes) que consumeixi recursos contínuament en un maquinari limitat, per què no delegar aquesta tasca al gestor de processos més madur i eficient que ja inclou Linux per defecte?

Aquí és on entra Systemd, el sistema d'inicialització estàndard de Linux. S'encarrega d'arrencar programes i reiniciar-los gairebé a l'instant si fallen. Històricament, integrar contenidors amb Systemd requeria l'enfocament *Legacy* (escriure complexos fitxers `.service`). Actualment, la solució recau en els *Quadlets* [3], una eina nativa de Podman que permet definir contenidors de manera declarativa. El sistema s'encarrega de traduir-ho automàticament a serveis, delegant el cicle de vida directament al *kernel*.

Ansible i la simulació de Kubernetes: Atès que Systemd i Podman només actuen a nivell de màquina individual, l'arquitectura es completa amb Ansible, una eina d'automatització descentralitzada sense agents instal·lats. En combinar aquestes tecnologies, es pot replicar l'estabilitat d'un orquestrador sencer sense pagar-ne l'"impost arquitectònic":

- **El Pla de Control (*Control Plane / Scheduler*):** Ansible actua com el cervell central. Mitjançant regles (*Playbooks*), decideix on, quan i com s'ha de desplegar l'aplicació a la flota de nodes.
- **L'Agent del Node (*Kubelet*):** Systemd substitueix el `kubelet`, assumint l'auto-recuperació. Si el procés cau, ho detecta a nivell de nucli i el reinicia localment.
- **El Motor d'Execució (*Container Runtime*):** Podman s'encarrega d'executar l'aplicació sense dependre de cap dimoni.
- **Els Manifestos Declaratius (YAML):** Els fitxers `Quadlet (.container)` substitueixen els complexos manifestos de Kubernetes, establint l'estat desitjat del contenidor.

2.4 Transmissió de Dades i Protocols de Comunicació

En entorns perifèrics, l'eficiència amb què es mouen les dades és tan important com com es processen.

Formats d'enviament de dades: La informació es pot estructurar en dos grans formats. D'una banda, el text pla (com JSON o XML), que és fàcilment llegible per humans però requereix empaquetar molta informació repetitiva i consumeix temps de CPU per ser processat. De l'altra, els formats binaris (com MessagePack o Protocol Buffers), que comprimeixen les dades a nivell de bits. Són il·legibles per a l'ull humà, però resulten extremadament lleugers i ràpids de serialitzar per les màquines, reduint la latència.

Protocols de comunicació:

- **HTTP/REST:** És l'estàndard web tradicional. Sol utilitzar format JSON, és fàcil d'implementar però pesat per a comunicacions massives degut a la càrrega de les capçaleres en cada petició.

- **gRPC**: Desenvolupat per Google, utilitza el protocol HTTP/2 i serialització binària (Protobuf). Permet múltiples crides simultànies i és molt eficient per a microserveis que necessiten respostes estructurades.
- **ZeroMQ**: Més que un protocol estàndard, és una llibreria de missatgeria d'alt rendiment. No utilitza un servidor intermediari (*brokerless*) i permet l'enviament asíncron directe de memòria a memòria, sent una de les opcions més ràpides i lleugeres quan es requereix processar milers de missatges per segon en xarxes locals.

2.5 Càrregues de treball exigents: La necessitat d'optimitzar per a la IA

L'aplicació pràctica d'aquestes optimitzacions es materialitza en el processament d'Intel·ligència Artificial al límit de la xarxa. Els models d'aprenentatge profund (*Deep Learning*) requereixen allotjar estructures de dades complexes (tensors) a la memòria RAM per a la seva execució. Qualsevol sobrecost d'infraestructura (com l'“Impost Arquitectònic” d'un orquestrador) sostreu recursos de computació directament al model d'IA, reduint-ne el rendiment.

Per quantificar aquest impacte de manera empírica, l'estudi requereix una càrrega de treball reconeguda i estandarditzada. Per això, el trànsit de xarxa se simula utilitzant el conjunt de dades MNIST, un estàndard clàssic de reconeixement de dígitos manuscrits que permet enviar un flux constant d'imatges als contenidors avaluats [4].

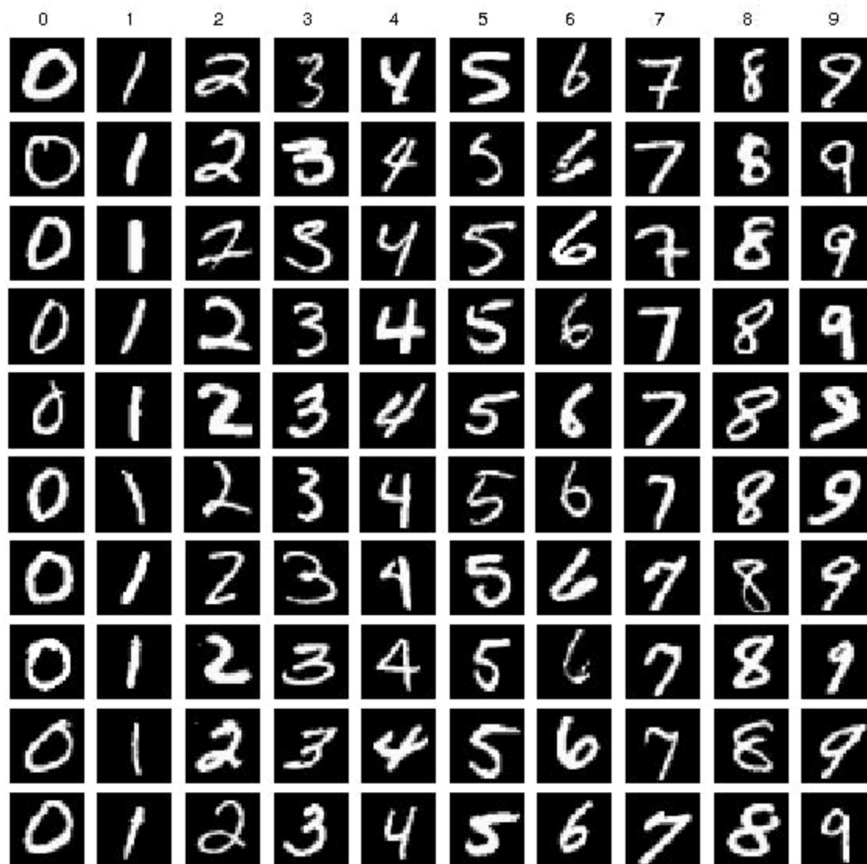


Figura 2.1: Datasets MNIST amb imatges de dígitos manuscrits classificats.

A nivell de programari d'IA, tot i que TensorFlow i PyTorch són els grans referents del mercat, s'ha optat per PyTorch [5]. La seva arquitectura basada en grafs computacionals dinàmics ofereix una

execució més àgil i un ús de memòria més controlable, unes característiques que s'alineen perfectament amb les restriccions de maquinari pròpies dels nodes *Edge*.

Finalment, l'arquitectura de la xarxa neuronal determina la intensitat del processament. Tot i que una opció inicial era utilitzar un simple Perceptró Multicapa (MLP), s'ha implementat una Xarxa Neuronal Convolucional (CNN). Aquesta decisió respon a la necessitat de generar un escenari realista de visió per computador: les complexes operacions matricials d'una CNN estressen de manera efectiva la CPU i la RAM del dispositiu. Això assegura que la prova no només mesuri l'enviament de dades, sinó que reproduceixi una càrrega de càlcul intensiva que reveli amb precisió com la penalització de l'orquestrador afecta la capacitat d'inferència de la màquina.

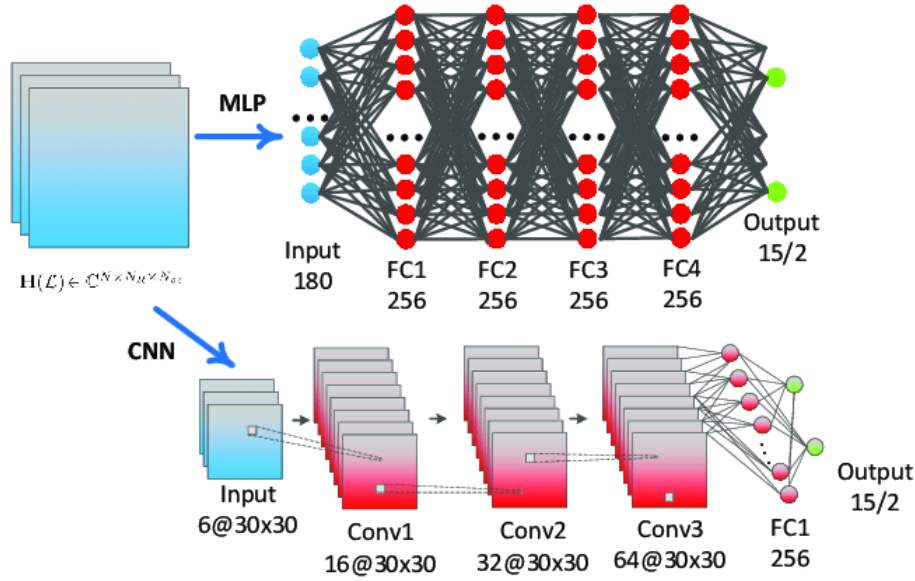


Figura 2.2: Arquitectura comú de xarxes MLP y CNN dissenyades per a classificació y regresió. Font: Adaptada de [1].

Capítol 3

State of the Art

A continuació s'exposa la revisió de l'estat de l'art organitzada segons les tendències actuals en la gestió de contenidors a l'Edge Computing, analitzant-ne les aportacions i limitacions.

3.1 Solucions d'Orquestració Lleugera a l'Edge (K3s, MicroK8s)

Gran part de la literatura recent aposta per adaptar l'estàndard de la indústria mitjançant distribucions reduïdes de Kubernetes com K3s [6]. Aquests treballs destaquen el manteniment de l'ecosistema declaratiu i l'escalabilitat horitzontal. No obstant això, estudis recents sobre consum de recursos a l'Edge assenyalen que aquests orquestradors continuen imposant un "Impost Arquitectònic" permanent a causa dels processos de control (com el kubelet o la xarxa SDN) [7].

3.2 Execució Nativa de Contenedors (*Standalone*)

D'altra banda, treballs previs plantegen prescindir de l'orquestració complexa i executar aplicacions mitjançant motors autònoms (Docker o Podman) per prioritzar la potència de càlcul [8]. Aquest enfocament maximitza el rendiment del maquinari en oferir accés a la xarxa nativa, però investigacions posteriors conclouen que se sacrifica dràsticament la tolerància a fallades i la capacitat de gestionar actualitzacions distribuïdes [9].

3.3 Avaluació de Protocols de Comunicació per a IA

En l'àmbit de la transmissió de grans volums de dades (com tensors d'IA), recerques recents comparen protocols com gRPC o ZeroMQ, identificant l'enviament asíncron amb serialització binària com la solució més eficient en latència [10]. Tanmateix, aquestes proves solen realitzar-se en entorns ideals, sense analitzar com l'encapsulament de paquets d'un orquestrador en segon pla pot arribar a degradar aquest rendiment.

3.4 El buit que omple aquest treball

Tot i que el recent paradigma de delegar el cicle de vida al *kernel* (utilitzant Podman integrat amb **Systemd** i orquestrat via **Ansible**) s'ha presentat com una solució tecnològica prometedora, fins avui hi ha una **falta de proves empíriques**: no existeixen estudis acadèmics ni proves de rendiment que hagin mesurat exactament quants recursos s'estalvien en aplicar aquesta arquitectura novedosa.

Aquest treball omple precisament aquest buit científic quantificant, per primera vegada i amb dades objectives, el sobrecost operatiu de Kubernetes. L'estudi no es limita a proposar teòricament l'ús de

Systemd, sinó que aporta la mètrica exacta del rendiment que es recupera en descartar l'orquestrador tradicional.

Avaluant empíricament ambdues plataformes sota càrregues d'estrès, el treball demostra amb xifres inèdites com l'alternativa nativa de Systemd minimitza el consum de memòria RAM en repòs, elimina el coll d'ampolla de les xarxes virtuals (SDN) per maximitzar el processament brut, i accelera l'auto-recuperació davant de fallades crítiques. D'aquesta manera, la investigació proporciona la justificació empírica que faltava a la indústria per validar Systemd com el substitut òptim de Kubernetes en escenaris d'Intel·ligència Artificial on cada megabyte compta.

Capítol 4

Methodology

Aquest capítol detalla el disseny experimental empíric creat per avaluar i quantificar les diferències de rendiment entre Kubernetes (K3s) i l'arquitectura nativa (Systemd). L'enfocament es basa en una comparativa A/B automatitzada per garantir la reproductibilitat de les dades.

4.1 Disseny Iteratiu i Selecció de Protocols

Abans de comparar les arquitectures, l'estudi es va plantejar de manera iterativa per identificar el protocol de xarxa més eficient i evitar que la transmissió de dades es convertís en el coll d'ampolla de les proves.

- **Fase 1: Exploració inicial de protocols (Entorn Nativa):** Inicialment, era necessari determinar l'impacte dels protocols de comunicació (HTTP, gRPC, ZeroMQ) i el format de les dades (text pla vers binari) en el rendiment. Utilitzant un entorn base amb Systemd i la càrrega de treball MNIST (el node worker fa el còmput de les imatges del dataset), es van mesurar mètriques exhaustives (*T_Total*, *Throughput*, *Tasa d'èxit*, *RTT*, *T_Proc*, *RAM_Max*, *CPU_Max* i *Payload_MB*). L'anàlisi inicial va evidenciar ràpidament que la llibreria ZeroMQ combinada amb serialització binària oferia el màxim rendiment.
- **Fase 2: El dilema de la serialització binària (Protobuf vers MessagePack):** Dins de l'ecosistema binari, va sorgir la necessitat d'avaluar diferents mètodes de serialització. Es va dissenyar una comparativa a quatre bandes: HTTP-JSON, gRPC-Protobuf, ZeroMQ-Protobuf i ZeroMQ-MessagePack. Tot i que Protobuf (inspirat en gRPC) té la reputació de ser extremadament ràpid, finalment es va optar per MessagePack a causa d'un problema tècnic específic amb les imatges del dataset MNIST:
 - *Sobrecàrrega de Protobuf:* Si la matriu de la imatge es defineix al fitxer `.proto` com una llista d'enters (`repeated int32`), la mida real del missatge es dispara de 784 bytes a més de 2 KB. Això consumeix una quantitat excessiva de CPU per empaquetar i desempaquetar cada píxel individualment. Tot i que es pot forçar l'ús del tipus `bytes` per mitigar-ho, fer-ho anul·la l'avantatge principal de Protobuf (el seu tipat estructurat fort).
 - *Eficiència de MessagePack:* Aquest format permet enviar les dades directament com un bloc binari cru (`bin` type) des de les estructures de memòria, evitant la penalització de CPU i mantenint l'empaquetatge extremadament lleuger.
- **Fase 3: Validació i selecció de l'orquestrador:** La decisió d'utilitzar ZeroMQ + MessagePack es va replicar i posar a prova en un entorn Kubernetes, obtenint-ne conclusions idèntiques pel que fa a l'eficiència del protocol. Per a la plataforma distribuïda, es va seleccionar explícitament K3s, ja que és la distribució certificada més lleugera de Kubernetes, dissenyada per minimitzar el consum de recursos a l'Edge.

- **Fase 4: Comparativa arquitectònica definitiva:** Un cop establert el protocol òptim (ZeroMQ-MessagePack) per garantir que la xarxa no fos el coll d'ampolla, i definit el programari distribuït (K3s), es va procedir a executar la comparativa d'estrès automatitzada final. Aquesta última fase és la que permet enfrontar en igualtat de condicions l'arquitectura nativa (Systemd) contra l'orquestrador (Kubernetes) per quantificar amb precisió l'“Impost Arquitectònic”.

4.2 Selecció del Model d'IA

Per garantir que la prova d'estrès generés una càrrega de processament d'imatges realista, es va descartar l'ús d'un Perceptró Multicapa (MLP) bàsic. L'estudi opta per una Xarxa Neuronal Convocucional (CNN), justificada per la seva superioritat en tasques de visió per computador en aprofitar la correlació espacial dels píxels. El disseny inclou blocs d'extracció de característiques i capes denses de classificació, generant matrius matemàtiques de pes suficient (tensors) per sotmetre la infraestructura a una inferència massiva. D'aquesta manera, el model actua com una càrrega de treball autèntica per avaluar el rendiment brut dels orquestradors a l'*Edge*, evitant introduir complexitat innecessària a l'arquitectura de la xarxa neuronal.

4.3 Topologia i Estratègia de Desplegament (Patró *Air-Gapped*)

L'estudi s'estructura mitjançant un model descentralitzat compost per un node de control central i diversos nodes perimetrals (*Workers*). Per evitar que les latències d'Internet o les descàrregues xarxa contaminin les mètriques de rendiment, s'adopta un patró *Air-Gapped* estricte:

- **Empaquetat centralitzat:** El programari es compila únicament al node mestre, alliberant els dispositius perimetrals de la pesada càrrega computacional.
- **Injecció local:** Els artefactes es transfereixen i s'injecten directament a la memòria local dels *Workers*.
- **Aïllament d'execució:** Tant en l'entorn natiu com en el distribuït, es força per disseny que la càrrega de treball recaigui exclusivament als nodes perimetrals, aïllant completament el pla de gestió.

4.4 Automatització i Reproduïbilitat (*Pipeline* d'execució)

Per garantir la validesa empírica de l'estudi i evitar biaixos manuals en la recollida de dades, s'ha dissenyat un *pipeline* d'execució completament automatitzat. El procés d'avaluació segueix un cicle estricte per a cada iteració de l'experiment:

- **Neteja i Desplegament:** S'esborra qualsevol rastre de recursos o memòria cau de les proves anteriors i es desplega la infraestructura completament des de zero. Això garanteix que cada arquitectura s'avalua sota un entorn idèntic i aïllat.
- **Estabilització i Refredament:** S'introdueixen pauses controlades entre desplegaments per evitar l'estrangulament tèrmic (*Thermal Throttling*) del maquinari perimetral. Això permet que els processos en segon pla de l'orquestrador s'estabilitzin completament abans de registrar el consum base d'operació.
- **Bucle d'Estrès:** S'executen múltiples iteracions consecutives de la prova de càrrega per mesurar el comportament de la infraestructura sota un estrès computacional sostingut.
- **Extracció al *Data Lake*:** Les mètriques obtingudes (consum, latència, resiliència i rendiment) s'extreuen dinàmicament dels registres en brut i s'aboquen automàticament a un fitxer estructurat centralitzat, actuant com a base de dades històrica immutable per a l'anàlisi posterior.

4.5 Tècniques de mesura i Mètriques d'Avaluació

L'avaluació arquitectònica es fonamenta en quatre mètriques clau d'infraestructura i resiliència a l'Edge. Totes les dades s'han extret de forma automatitzada mitjançant scripts de monitoratge cap a un Data Lake (CSV) per garantir el rigor estadístic i evitar biaixos manuals:

- **Temps de Desplegament (T_{deploy}):** Mesura en segons el temps total necessari per aprovisionar els nodes, configurar l'orquestrador (imperatiu vs declaratiu) i carregar els artefactes de programari (imatges de contenidors i pesos del model) abans d'iniciar la inferència.
- **Consum de CPU en Repòs (CPU_Reposo):** Quantifica el percentatge de processament que monopolitzen els dimonis de gestió subjacents (com el *kubelet* o el *containerd*) quan el sistema no rep trànsit, evidenciant la càrrega constant sobre el processador.
- **Consum Base de RAM (RAM_Reposo o Impost Arquitectònic):** Mesura de la memòria ocupada en repòs just abans d'injectar la càrrega de treball. Serveix per quantificar el sobrecost exclusiu de la plataforma i dictaminar-ne la viabilitat operativa en dispositius perimetrals amb recursos estrictament limitats (mesurat en MB).
- **Resiliència i Enginyeria del Caos (MTTR):** Càlcul del Temps Mitjà de Recuperació forçant una eliminació abrupta dels contenidors actius. Es mesura en mil·lisegons el temps exacte transcorregut des de l'ordre de destrucció (fallada crítica) fins que el sistema orquestrador aconsegueix aixecar un nou servei i torna a estar llest per rebre peticions.

4.5.1 Tractament de Dades (Data Cleansing)

Durant l'execució de les proves de resiliència en Kubernetes, es va registrar un valor atípic extrem (*Outlier*) en una de les 5 iteracions, on el sistema va trigar 261.450 ms (aproximadament 4 minuts) a recuperar-se a causa d'un bloqueig temporal (*lock*) en l'alliberament de la IP virtual per part del connector de xarxa Flannel. Per garantir l'equitat i no alterar la mida de la mostra estadística, no s'ha descartat aquesta execució. En el seu lloc, s'ha aplicat una tècnica d'imputació de la mitjana, substituint aquest error d'infraestructura puntual pel valor mitjà de les altres n execucions estables. Aquesta correcció permet mantenir el volum de dades original evitant que una anomalia distorsioni el comportament general de l'orquestrador.

Capítol 5

Implementation

Aquest capítol detalla la implementació tècnica de l'entorn de proves, descrivint l'arquitectura de programari, les eines de desenvolupament utilitzades i els procediments d'automatització dissenyats per extreure dades comparatives fiables.

5.1 Tecnologies, Llenguatges i Eines Utilitzades

- **Ansible:** Eina central per orquestrar la preparació de la infraestructura en ambdós entorns. Permet la injecció dinàmica de variables (com `experimento_path`) per avaluar múltiples protocols de xarxa de manera desatesa.
- **Podman i Systemd:** Tecnologies base per a l'arquitectura nativa. Podman construeix les imatges ajustant el gestor de recursos (`--cgroup-manager=cgroupfs`) i les executa en mode *rootless*. El cicle de vida es delega a fitxers declaratius *Quadlet* gestionats per Systemd.
- **K3s i kubect1:** Distribució lleugera de Kubernetes utilitzada com a referència d'orquestrador perimetral.
- **Python (PyTorch i Lògica de Xarxa):** Llenguatge principal de programació del projecte. S'ha utilitzat per desenvolupar tota la lògica dels nodes mestre i treballador (*Master/Worker*), i per implementar els diferents protocols de comunicació avaluats (HTTP-JSON, gRPC, ZeroMQ). A més, s'ha utilitzat programació en Python per automatitzar la descàrrega del conjunt de dades MNIST del núvol i per dissenyar, entrenar i exportar el model d'Intel·ligència Artificial mitjançant la llibreria especialitzada PyTorch.
- **Bash:** Llenguatge utilitzat per construir els *scripts* d'automatització que governen l'estrès de la infraestructura, l'enginyeria del caos i la invocació repetitiva de les proves de xarxa.

5.2 Disseny i Entrenament del Model d'IA (PyTorch)

Per garantir que la prova d'estrès generés una càrrega de processament d'imatges realista abans de l'avaluació de la infraestructura, es va dissenyar i entrenar un model d'Intel·ligència Artificial personalitzat amb el *dataset* MNIST (80% per a entrenament, 20% per a validació), descartant l'ús d'un Perceptró Multicapa (MLP) bàsic en favor d'una Xarxa Neuronal Convolucional (CNN).

- **Justificació de l'Arquitectura:** La decisió d'optar per una CNN respon a la seva superioritat en tasques de visió per computador, ja que permet aprofitar la correlació espacial dels píxels de les imatges.
- **Bloc d'Extracció de Característiques:** Està compost per dues capes convolucionals (Conv2d) que extreuen 32 i 64 mapes de característiques, respectivament. Cada convolució va seguida

d'una funció d'activació no lineal (ReLU) i d'una capa d'agrupació (MaxPool2d) per reduir la dimensionalitat i el cost computacional.

Listing 5.1: Crear un model de Xarxa Neuronal Convolucional (CNN) amb PyTorch.

```

1 class CNN(nn.Module):
2     def __init__(self):
3         super(CNN, self).__init__()
4         self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1)
5         self.relu = nn.ReLU()
6         self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
7
8         self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)
9         self.fc1 = nn.Linear(64 * 7 * 7, 128)
10        self.fc2 = nn.Linear(128, 10)
11
12    def forward(self, x):
13        x = self.pool(self.relu(self.conv1(x)))
14        x = self.pool(self.relu(self.conv2(x)))
15        x = x.reshape(-1, 64 * 7 * 7)
16        x = self.relu(self.fc1(x))
17        x = self.fc2(x)
18        return x
19
20 # Initialize the CNN model
21 model = CNN()
22 print(model)

```

- **Bloc de Classificació:** Un cop aplanats els mapes de característiques mitjançant l'operació reshape, la informació es transmet a través de dues capes denses (*Fully Connected* o Linear). Aquestes capes redueixen progressivament les dimensions fins a produir una sortida final de 10 nodes, que representen les probabilitats d'encert per als 10 dígitos possibles (del 0 al 9).
- **Paràmetres d'Entrenament (MLOps):** L'optimització dels pesos de la xarxa neuronal s'ha dut a terme aplicant l'algoritme de Descens de Gradient Estocàstic (SGD), configurat amb una taxa d'aprenentatge de 0.01 i un impuls (*momentum*) de 0.9. La funció de cost seleccionada ha estat l'Entropia Creuada (CrossEntropyLoss), i s'ha establert un únic cicle d'entrenament (1 època) processant les imatges en blocs (*batch size*) de 64 unitats.

Justificació de l'Entrenament d'1 Època:

Per a l'avaluació del sistema, l'entrenament de la CNN es va configurar deliberadament per executar-se durant una única època (1 Epoch). Aquesta decisió arquitectònica es fonamenta en tres criteris clau:

1. **Abast del Projecte:** L'objectiu principal d'aquest estudi és la comparativa de systemd amb kubernetes a l'entorn *Edge*, no l'optimització d'hiperparàmetres en models de *Deep Learning*.
2. **Evidència de Convergència:** Com s'observa a la corba d'aprenentatge, l'error d'entrenament (*Train Loss*) experimenta una caiguda dràstica i s'estabilitza abans de finalitzar la primera iteració completa del *dataset*.
3. **Eficiència Computacional:** En finalitzar aquesta única època, el model assoleix una precisió de validació (*Validation Accuracy*) propera al 98%. Aquest nivell d'encert és prova empírica suficient per certificar que el *pipeline* d'inferència i la transmissió de tensors funcionen correctament d'extrem a extrem, evitant un consum innecessari de temps i recursos al node Orquestrador.

5.3 Infraestructura com a Codi: Arquitectura Nativa (Systemd)

Aquesta secció detalla la Infraestructura com a Codi (IaC) basada en Ansible per a l'aprovisionament automatitzat i la gestió del cicle de vida dels nodes (Mestre i *Workers*) en l'entorn de processament perimetral natiu.

Arquitectura i Implementació Tècnica

El sistema està dissenyat sota el paradigma de contenidors *rootless* per maximitzar la seguretat perimetral, delegant el control dels serveis directament a Systemd:

- **Systemd Linger:** S'habilita la persistència de processos en segon pla per a l'usuari del sistema (`littledragon`) executant `loginctl enable-linger`. Això garanteix que els serveis es mantinguin actius un cop tancada la sessió SSH.
- **Podman Quadlets:** S'elimina l'ús tradicional de `docker-compose`. L'estat desitjat es declara mitjançant Quadlets, injectant plantilles Jinja2 directament a `~/.config/containers/systemd/` per generar xarxes (`red.network`) i contenidors (`worker.container`) de forma nativa.
- **Aïllament i Alta Disponibilitat:** El sistema crea una xarxa virtual dedicada (`red-interna`). A més, la unitat de Systemd aplica polítiques d'auto-recuperació (`Restart=always` amb 3 segons d'espera) per garantir la resiliència del *Worker* davant de fallades.

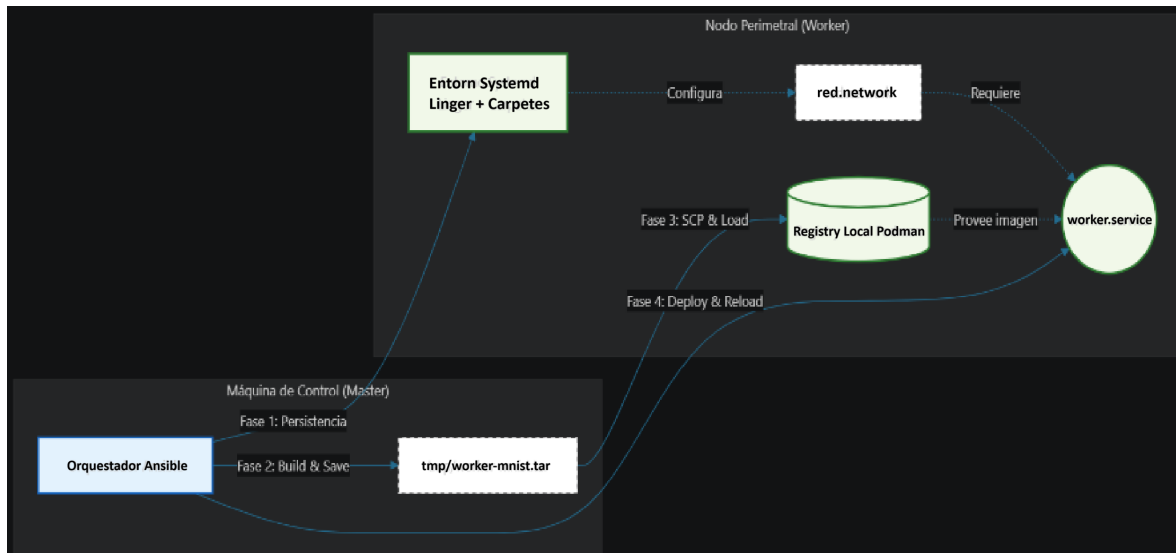


Figura 5.1: Arquitectura de l'entorn nativa amb Systemd.

Disseny Modular i Desplegament Dinàmic (*Air-Gapped*)

El *pipeline* de desplegament és completament independent del protocol de comunicació. La injecció de l'entorn es controla dinàmicament en temps d'execució sobrescrivint la ruta de l'experiment amb variables addicionals (`-e "experimento_path=..."`), permetent pivotar l'arquitectura sense alterar el codi font. S'utilitza un patró *Air-Gapped*:

- Ansible itera sobre el protocol seleccionat i compila la imatge OCI centralitzadament al node de control.
- La imatge s'empaqueta en un artefacte immutable (`.tar`) i es transfereix via SSH.
- Es carrega directament al motor local (`podman load`), alliberant l'*Edge* de la descàrrega de dependències.

Seguretat i Aprovisionament Desatès (*Zero-Touch Provisioning*)

Per assolir una automatització absoluta sense intervenció humana i protegir el repositori de codi, l'escalada de privilegis (sudo) s'injecta des del fitxer local `group_vars/workers.yml` (exclòs via `.gitignore`), eliminant la necessitat d'introduir contrasenyes interactives (-K) durant el desplegament.

Documentació d'Execució i Benchmarking

El procés està governat pels següents components d'automatització:

- **playbook.yml:** Activa el *Linger*, desplega la xarxa, distribueix l'artefacte `.tar`, instància el *Quadlet* i habilita el `worker.service`.
- **clean.yml:** Executa una purga agressiva aturant els serveis i forçant l'esborrat de xarxes internes per evitar interferències entre proves.
- **generate_benchmarks.sh:** Orquestra el cicle d'avaluació executant el desplegament, capturant el consum base de RAM/CPU, aplicant Enginyeria del Caos (`podman kill`) per mesurar el MTTR i, finalment, llançant la simulació d'inferència per consolidar les mètriques en un *Data Lake* CSV.

5.4 Infraestructura com a Codi: Arquitectura Distribuïda (K3s)

Aquesta secció detalla l'adaptació de l'entorn cap a un model declaratiu i centralitzat utilitzant K3s, una distribució lleugera de Kubernetes optimitzada per a l'*Edge* (IoT/ARM) [11].

Arquitectura i Implementació Tècnica

- **Motor de Contenidors (containerd):** Se substitueix Podman pel motor intern de Kubernetes. La gestió d'imatges queda sota el control absolut del component `Kubelet` de cada node.
- **Manifestos Declaratius (YAML):** S'elimina l'ús de *Quadlets*. L'estat es defineix al directori `k8s-manifests/`, aplicant la configuració al clúster via `kubectl apply`.
- **Xarxa Definida per Programari (SDN):** L'aïllament i la connectivitat depenen de la xarxa virtual del clúster (CNI Flannel), permetent la comunicació transparent entre nodes.
- **Resiliència i Auto-Recuperació:** El bucle de reconciliació (*Reconciliation Loop*) de Kubernetes garanteix que l'estat real coincideixi amb el declarat, aixecant nous *Pods* automàticament davant de caigudes.

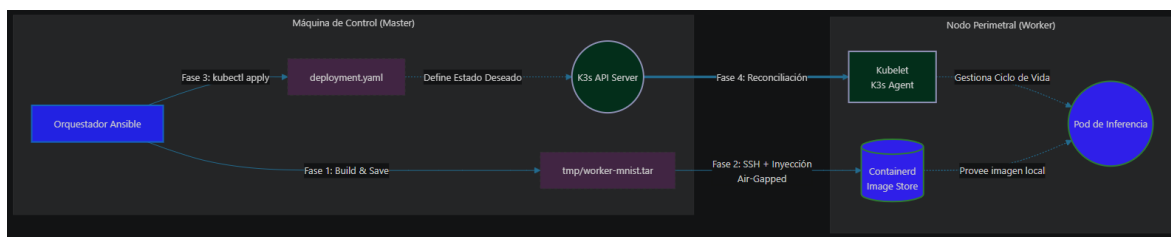


Figura 5.2: Arquitectura de l'entorn nativa amb Kubernetes.

Disseny Modular i Desplegament Dinàmic (*Air-Gapped*)

L'arquitectura manté l'enfocament modular i dinàmic (-e "experimento_path=...") sota el patró *Air-Gapped* adaptat a K3s:

- Compilació de la imatge OCI al node mestre.
- Empaquetament en un artefacte `.tar` i transferència via SSH.

- Injecció directa al *namespace* intern de K3s utilitzant `k3s ctr images import`, evitant dependre de registres externs.

Seguretat i Inventari

Igual que a l'entorn natiu, la topologia es defineix a l'`inventory.ini` separant el grup `[master]` (execució local per optimitzar la compilació) del grup `[workers]`. La seguretat es gestiona mitjançant credencials aïllades a `group_vars/workers.yml` per facilitar un *Zero-Touch Provisioning*. Els serveis s'exposen a través d'un *Service* tipus *NodePort* pel port 8000.

Documentació d'Execució i Benchmarking

- **playbook.yml:** S'encarrega de la transferència *Air-Gapped*, injecta els manifestos YAML al clúster i delega al Kubelet la inicialització fins a assolir l'estat *Ready*.
- **clean.yml:** Executa una purga lògica eliminant els recursos descrits als manifestos i neteja la memòria cau d'imatges amb `k3s crictl rmi` per evitar la contaminació de mètriques (processos *zombie*).
- **generate_benchmarks.sh:** Mapeja dinàmicament el nom lògic del node en K3s a la seva IP física per capturar el consum global pur. Força l'eliminació abrupta d'un *Pod* (`kubectl delete pod --force`) per mesurar el MTTR monitoritzant el bucle de reconciliació, i executa la prova d'estrès bolcant les dades mitjançant *Regex* al CSV consolidat.

5.5 Disseny del Protocol de Comunicació i Inferència d'IA

Un cop desplegada la infraestructura, la transmissió de les dades s'efectua mitjançant una arquitectura de missatgeria síncrona *Request-Reply* (REQ-REP) basada en TCP pur amb ZeroMQ. Després d'avaluar diverses alternatives de serialització binària, es va descartar Protobuf en favor de MessagePack per optimitzar l'agilitat operativa en l'ecosistema d'aprenentatge automàtic.

Aquesta implementació elimina la necessitat de precompilar contractes estrictes (`.proto`), adoptant un format *schema-less* (sense esquema) que s'integra nativament amb els diccionaris de Python i distribueix la càrrega entre el Pla de Control i el Pla de Dades, tal com s'il·lustra a les figures globals de l'arquitectura:

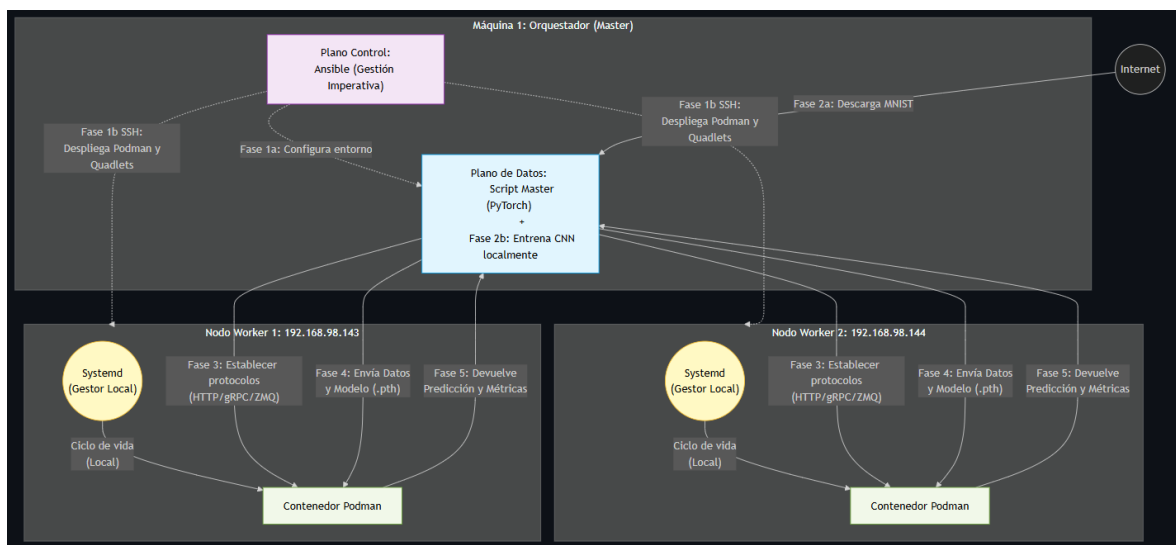


Figura 5.3: Topologia completa de la inferència d'IA sobre Systemd.

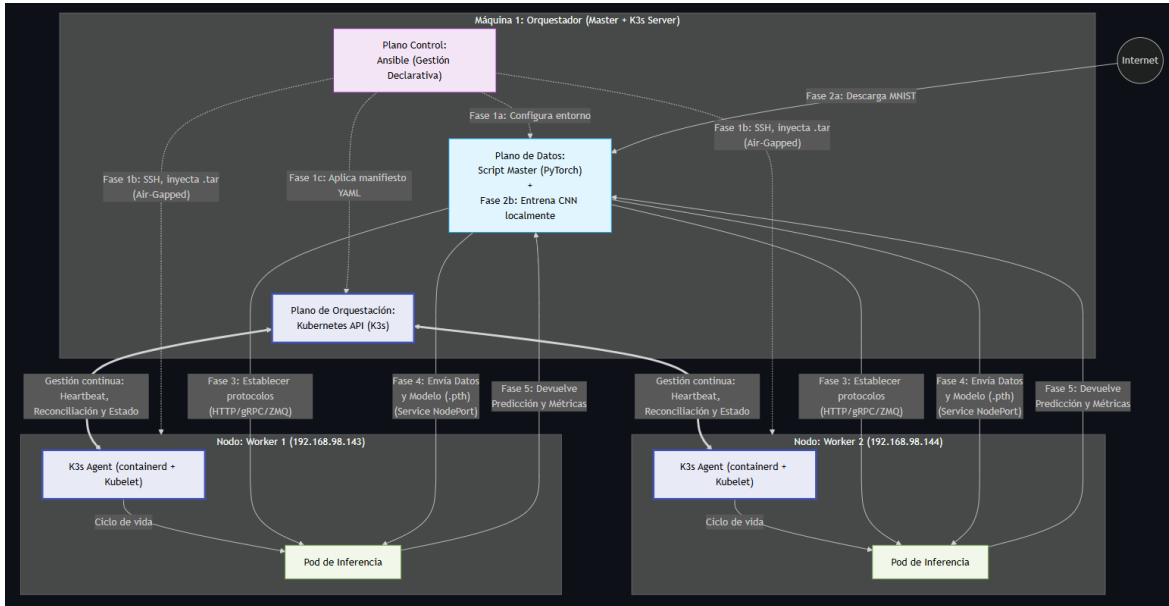


Figura 5.4: Topologia completa de la inferència d'IA sobre K3s.

El flux d'execució i l'enrutament s'estructuren de la següent manera:

- **Orquestració i Estructuració Dinàmica (Node Mestre):** El mestre assumeix el control total de l'entrenament (*Offline*) i la injecció de dades. En lloc d'instanciar classes precompilades, empaqueta els bytes del model (.pth) i els tensors d'imatges directament en un diccionari estàndard ({'model_data': model_bytes}). Aquesta estructura es comprimeix dinàmicament a format binari utilitzant `msgpack.packb(..., use_bin_type=True)` i s'injecta a la xarxa mitjançant missatgeria multiparte.
- **Gestió de Memòria i Seguretat (Nodes Workers):** Atès que les llibreries de MessagePack imposen límits estrictes de memòria per prevenir atacs per esgotament (OOM), el servidor perimetral sobrescriu explícitament aquests límits (`max_bin_len=200*1024*1024`) per permetre la deserialització de tensors pesats propis de PyTorch.
- **Inferència i Còpia Zero (Zero-Copy):** En rebre la petició d'inferència, el Worker recupera el tensor mitjançant la clau del diccionari i aplica tècniques de còpia zero (`np.frombuffer`). Això permet assignar el bloc de memòria directament a la Xarxa Neuronal Convolucional (CNN), invertint tot l'esforç de la CPU en el càlcul matemàtic (`torch.no_grad()`) sense duplicar les dades a la RAM.
- **Enrutament segons l'Arquitectura:** En l'entorn natiu (Systemd), el diccionari MessagePack s'envia directament al port local del contenidor. En l'entorn distribuït (K3s), el mestre injecta els paquets a través d'un servei *NodePort* exposat al port 30000, delegant a l'orquestrador l'enrutament intern cap al *socket* del *Pod* corresponent.

5.6 Automatització del Pipeline i Data Lake

L'avaluació es fonamenta en una jerarquia de tres *scripts* de Bash que actuen com a fil d'execució continu per evitar qualsevol biaix manual. En el nivell superior, un **orquestrador mestre híbrid** governa el cicle de vida global de l'experiment. La seva funció principal és gestionar la transició entre plataformes i garantir la integritat del maquinari, forçant reinicis físics (`sudo reboot`) als nodes perimetrals i aplicant pauses de refredament de 120 segons per evitar la degradació del rendiment per sobreescalfament (*Thermal Throttling*).

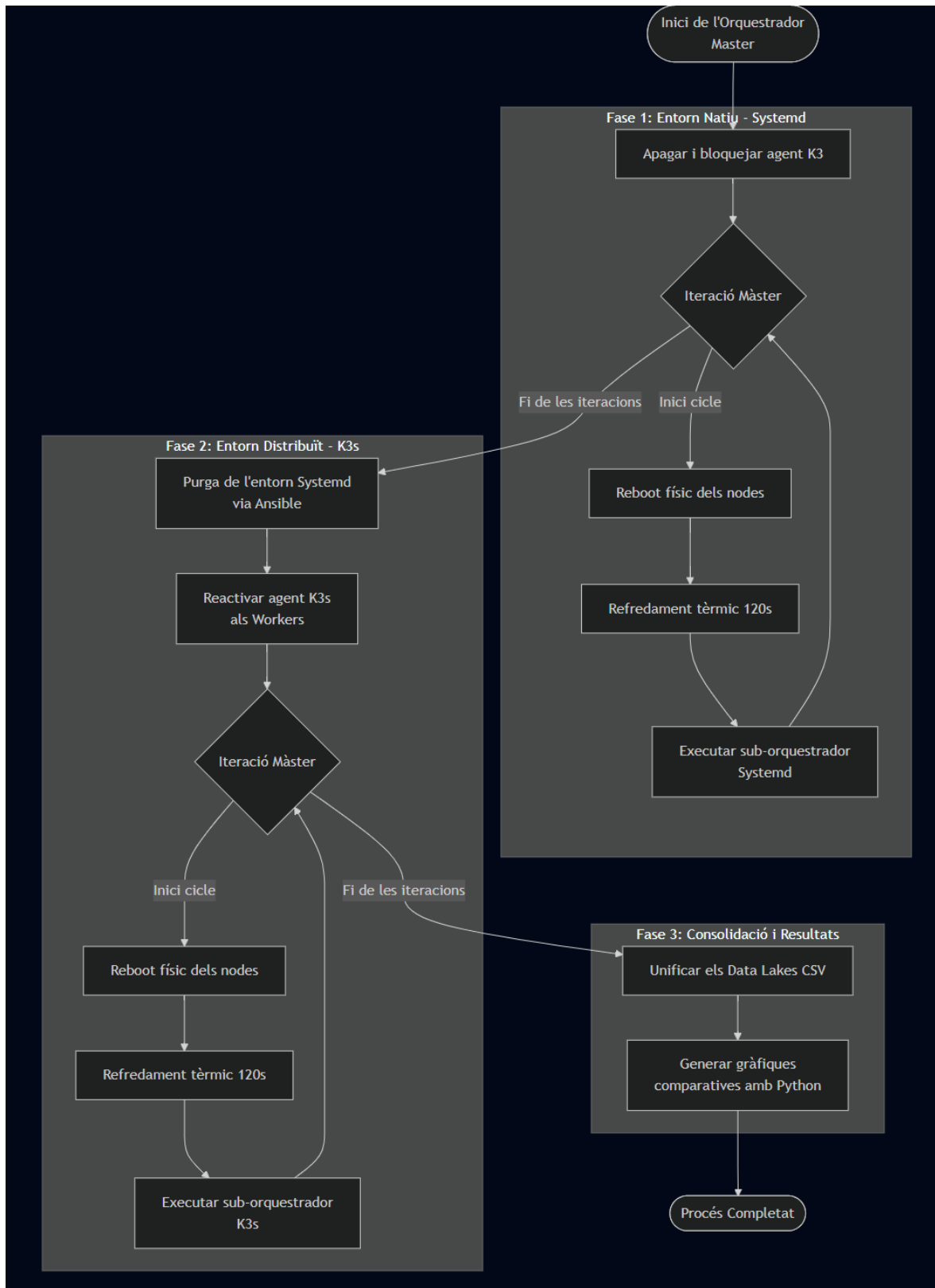


Figura 5.5: Flux del pipeline d'execució automatitzada.

Un cop l'entorn físic està estabilitzat, aquest orquestrador mestre invoca els *scripts* específics d'avaluació (un dissenyat exclusivament per a Systemd i un altre per a K3s). Aquests sub-orquestradors iteren seqüencialment sobre els protocols executant les següents fases:

- **Inicialització i Estabilització:** S'executa una neteja total de l'entorn previ i es desplega la infraestructura des de zero. S'apliquen pauses d'estabilització (de 15 segons en K3s) per permetre que els processos en segon pla com el *metrics-server* s'assentin i recullin dades fiables del consum base.
- **Enginyeria del Caos:** Destrucció abrupta del procés actiu i mesura mil·limètrica del Temps Mitjà de Recuperació (MTTR) mitjançant consultes iteratives a l'estat del sistema.
- **Bucle d'Estrès i Extracció (Data Lake):** Execució de 5 llançaments consecutius del client Python per saturar la xarxa. Un cop finalitzats, un *script* de Python incrustat dins de Bash analitza els registres en text pla mitjançant expressions regulars (Regex) i bolca dinàmicament totes les mètriques estructurades (latència, *Throughput*, memòria màxima, etc.) al fitxer consolidat `resultados_globales.csv`.

Listing 5.2: Extracció de mètriques Regex cap al Data Lake CSV incrustada en Bash

```

1 # Extracció de Infraestructura
2 t_deploy = get_avg(log_file, r"T_deploy total:\s*([0-9\.]+)")
3 cpu_reposo = get_avg(log_file, r"CPU Reposo \(\Global\):\s*([0-9\.]+)%")
4 # Podman suele devolver '15.2MB' o '15.2MiB', el regex coge el 15.2 perfecto
5 ram_reposo = get_avg(log_file, r"RAM Reposo \(\Global\):\s*([0-9\.]+)")
6 mttr = get_avg(log_file, r"Tiempo Real Arranque:\s*([0-9\.]+)")
7
8 # Extracció de Red (Añadida la Tasa de Éxito)
9 t_total = get_avg(log_file, r"Tiempo Total \(\s\):\s*([0-9\.]+)")
10 throughput = get_avg(log_file, r"Throughput \(\img/s\):\s*([0-9\.]+)")
11 # Regex tolerante: Atrapa "Tasa Éxito" o "Tasa de Éxito"
12 tasa_exito = get_avg(log_file, r"Tasa(?: de)? Éxito \(\%\):\s*([0-9\.]+)")
13 rtt = get_avg(log_file, r"Latencia RTT Promedio \(\ms\):\s*([0-9\.]+)")
14 t_proc = get_avg(log_file, r"Tiempo T_proc Promedio \(\ms\):\s*([0-9\.]+)")
15 ram_max = get_avg(log_file, r"Pico Máx\ RAM Worker \(\MB\):\s*([0-9\.]+)")
16 cpu_max = get_avg(log_file, r"CPU Promedio \(\%\):\s*([0-9\.]+)")
17 payload_nodo = get_avg(log_file, r"Payload Promedio por Nodo \(\MB\):\s*([0-9\.]+)")
18
19 # Escribir en CSV
20 file_exists = os.path.isfile(csv_file)
21 with open(csv_file, 'a', encoding='utf-8') as f:
22     if not file_exists:
23         f.write("Fecha,Protocolo,T_deploy,CPU_Reposo,RAM_Reposo,MTTR,T_Total,Throughput,Exito_
24             %,RTT,T_Proc,RAM_Max,CPU_Max,Payload_MB\n")
25
26     f.write(f"{fecha},{protocol_name},{t_deploy:.2f},{cpu_reposo:.2f},{ram_reposo:.2f},{mttr
27         :.2f},{t_total:.2f},{throughput:.2f},{tasa_exito:.2f},{rtt:.2f},{t_proc:.2f},{ram_max
28         :.2f},{cpu_max:.2f},{payload_nodo:.2f}\n")

```

5.7 Decisions Tècniques i Resolució de Problemes

La mesura precisa del MTTR va requerir solucions adaptades a cada tecnologia. Per a l'enfocament natiu, la fallada se simula amb un `podman kill` i la detecció de recuperació s'aconsegueix amb bucles `while` de 0.05 segons interrogant l'estat local del `systemctl`. Per contra, en l'entorn distribuït s'ha de fulminar el procés forçant el tancament (`kubectl delete pod --force --grace-period=0`) i esperar que el *DaemonSet* assigni i validi el nou *Pod* monitoritzant el seu canvi a l'estat *Ready*.

Durant la fase d'enginyeria del caos en l'entorn distribuït, es va identificar una incidència crítica d'infraestructura: un bloqueig (*lock*) en l'alliberament de la IP virtual per part del connector Flannel, fet que va disparar el temps de recuperació d'una iteració a 261.450 ms (aproximadament 4 minuts).

Per salvaguardar el rigor estadístic general, aquest valor atípic extrem es va descartar aplicant una tècnica d'imputació de la mitjana (*Data Cleansing*), evitant que un error transitori de la subxarxa SDN virtual distorsionés l'avaluació global de l'orquestrador.

Capítol 6

Results

6.1 Selecció del Protocol de Comunicació

Abans d'avaluar l'impacte arquitectònic dels orquestradors, és imperatiu determinar quin és el protocol de xarxa òptim per a la transmissió massiva de tensors matemàtics. L'objectiu d'aquesta fase és aïllar la capa de transport per garantir que la serialització de les dades no actuï com a coll d'ampolla. Totes les proves d'aquesta secció s'han executat exclusivament sobre l'entorn natiu (Systemd) per registrar el límit físic del maquinari sense interferències de xarxes virtuals.

La Taula 6.1 presenta les mètriques consolidades, extretes de la mitjana històrica del *Data Lake* després de sotmetre els nodes a càrrega extrema.

Protocol (Systemd)	T_{Total} (s)	Throughput (img/s)	RTT (ms)	T_{proc} Worker (ms)	RAM Màx (MB)	Payload (MB)
HTTP / JSON	0.80	2492.71	416.45	402.00	329.12	15.59
gRPC / Protobuf	0.23	8621.28	223.62	198.63	311.62	2.99
ZeroMQ / Protobuf	0.22	9299.01	212.55	197.79	275.61	2.99
ZeroMQ / MessagePack	0.22	9079.08	214.71	198.58	280.28	2.99

Taula 6.1: Resultats consolidats de rendiment de xarxa per protocol a l'Edge.

De l'anàlisi d'aquestes mètriques i la seva representació visual, s'extreuen les següents conclusions:

1. El Coll d'Ampolla del Text Pla (HTTP/JSON) L'estàndard web tradicional es demostra totalment inviable per al processament perimetral. En no suportar transmissió binària nativa, els tensors es converteixen a text pla, inflant el *payload* fins a 15,59 MB per petició. Com s'observa a la Figura 6.1, el seu rendiment queda severament estancat (2.492 img/s). A més, la mètrica del T_{proc} (402 ms) evidencia que la CPU del *Worker* col·lapsa intentant parsejar documents de text gegantins en lloc de calcular prediccions d'IA.

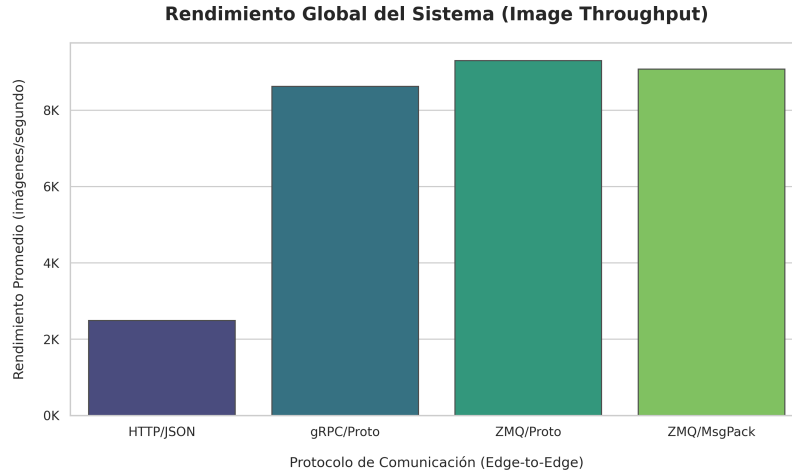


Figura 6.1: Comparativa de rendiment brut (*Throughput*) processant tensors.

2. El Salt a la Serialització Binària (gRPC) La transició cap a gRPC i Protobuf resol els problemes crítics del *baseline*. La càrrega de xarxa es redueix al seu pes matemàtic pur (2,99 MB). Aquesta eficiència permet utilitzar tècniques de *Zero-Copy* (np.frombuffer) a la memòria cau, reduint el temps de processament local a la meitat (198 ms) i disparant el *throughput* per sobre de les 8.600 img/s.

3. Eficiència de Memòria i el Límit del Transport (ZeroMQ) Tot i l'alta velocitat de gRPC, el seu motor intern basat en HTTP/2 afegeix un *overhead* de capçaleres innecessari. Substituir-lo per connexions TCP crues (*Brokerless*) dengen ZeroMQ assoleix el límit físic de velocitat. Tal com s'il·lustra a la Figura 6.2, les architectures basades en ZeroMQ presenten la petjada de memòria (*RAM Footprint*) més baixa sota estrès, aconseguint un estalvi crític per a dispositius *Edge* limitats en comparació amb l'ecosistema gRPC.

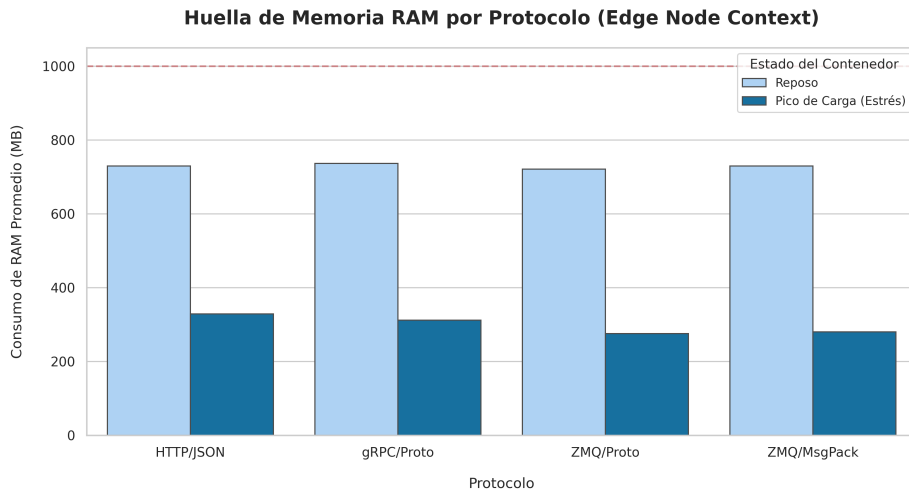


Figura 6.2: Impacte de la serialització en el consum màxim de memòria RAM als *Workers*.

4. Matriu de Trade-off La decisió arquitectònica definitiva es visualitza a la matriu de *Trade-off* (Figura 6.3), que encreua la capacitat de processament global amb la latència estructural. El quadrant superior esquerre (alta velocitat i mínima latència) és dominat absolutament per ZeroMQ.

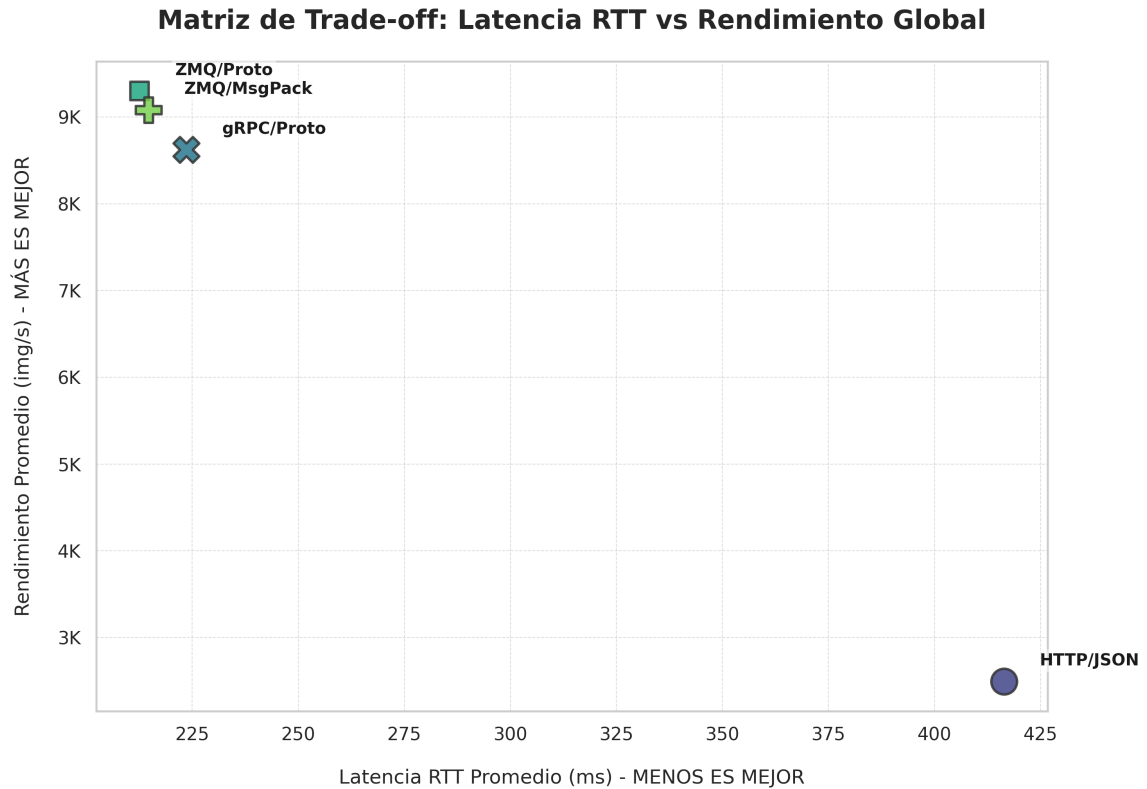


Figura 6.3: Relació de compromís (*Trade-off*) entre rendiment global i Latència RTT.

Dins d'aquest quadrant líder, **ZeroMQ combinat amb MessagePack** es consolida com l'arquitectura guanyadora. Tot i presentar un rendiment marginalment inferior a Protobuf (9.079 vs 9.299 img/s), MessagePack aporta un format *schema-less* que s'integra nativament amb estructures de Python. Això elimina la complexitat de precompilar contractes estrictes (.proto), estabilitzant el pic de memòria cau i maximitzant l'agilitat operativa per a fluxos MLOps. Aquesta combinació s'estableix com l'estàndard de transport definitiu per avaluar els orquestradors a la resta de l'estudi.

6.2 Rendiment del Model d'IA (CNN)

Per garantir que la infraestructura se sotmet a una càrrega d'inferència realista abans de comparar els orquestradors, és imprescindible certificar la viabilitat operativa i la integritat de les dades en el sistema distribuït. S'ha validat l'entrenament del model utilitzant el protocol guanyador (ZeroMQ amb MessagePack), demostrant que la partició del *dataset* i la transmissió de tensors es realitza sense corrompre el *payload*.

L'anàlisi de les mètriques visuals extretes durant la fase d'entrenament (*Offline*) confirma el comportament esperat de la Xarxa Neuronal Convolucional (CNN):

- **Convergència de l'Error (Corba de Pèrdua):**

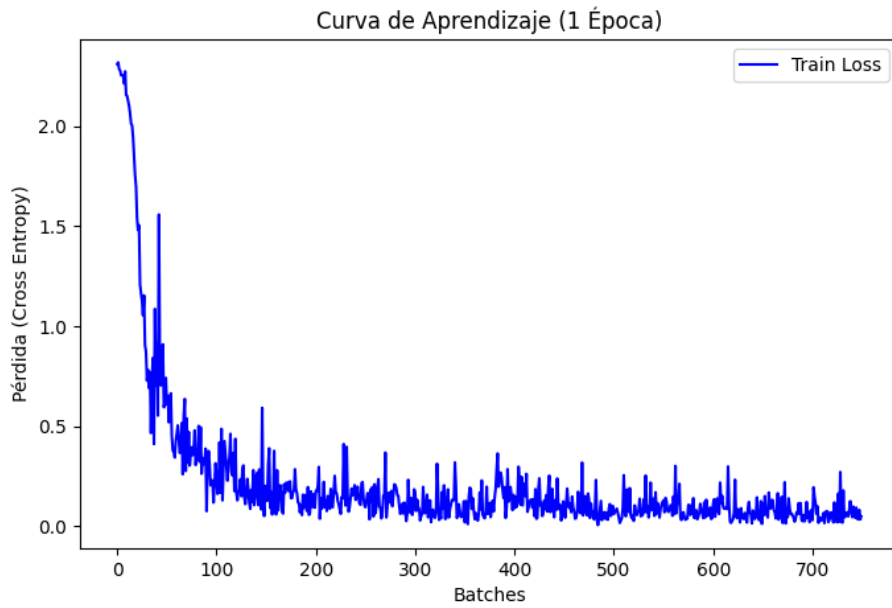


Figura 6.4: Avaluació del model fundacional: Corba de pèrdua (*Train Loss*).

Com s'observa a la Figura 6.4, l'error d'entrenament experimenta una caiguda dràstica i s'estabilitza abans de finalitzar la primera iteració completa del *dataset*. Això justifica empíricament la decisió d'entrenar durant una única època (1 Epoch), evitant un consum innecessari de temps al node Mestre.

- **Precisió (Matriu de Confusió):**

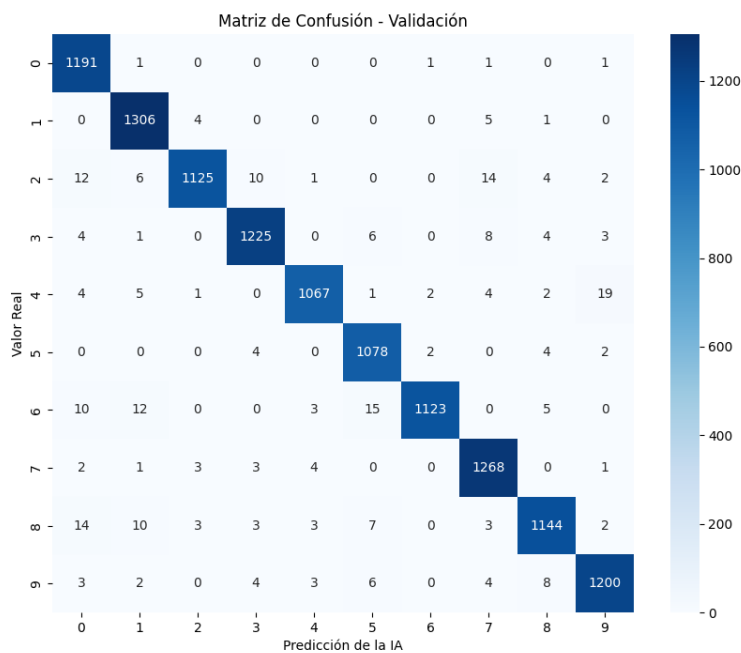


Figura 6.5: Avaluació del model fundacional: Matriu de confusió en validació.

La diagonal intensament fosca de la Matriu de Confusió (Figura 6.5) demostra una alta taxa d'encerts (Veritables Positius), assolint una precisió de validació propera al 98%. La baixa dispersió fora de la diagonal certifica que el model no pateix sobreajustament (*overfitting*).

• Integritat de la Transmissió (Exemples Visuals):

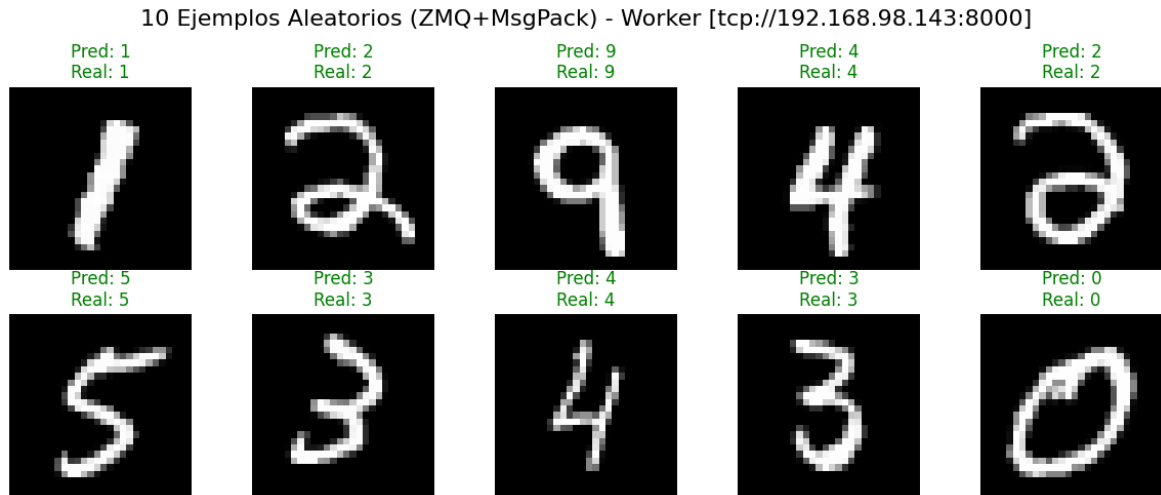


Figura 6.6: Exemples d'inferència del model fundacional: predicció i valor real al node Worker.

Les mostres aleatòries capturades directament des del *Worker* (Figura 6.6) verifiquen que la serialització dinàmica amb `msgpack` reconstrueix la imatge perfecta a l'altre extrem de la xarxa, obtenint inferències lògiques i correctes.

Amb la fiabilitat algorítmica garantida, el model actua com una eina d'estrès realista per començar la comparativa definitiva d'infraestructures.

6.3 Comparativa Arquitectònica a l'Edge: Systemd vs Kubernetes (K3s)

Aquesta secció presenta la conclusió tècnica del projecte, enfrontant l'arquitectura nativa basada en Systemd front a l'orquestrador modern Kubernetes (K3s). Per garantir una avaluació equitativa, ambdues plataformes s'han sotmès a proves d'estrès utilitzant el protocol de xarxa més eficient identificat prèviament: **ZeroMQ amb serialització MessagePack**.

L'objectiu d'aquesta comparativa és quantificar l'Ímpuls Arquitectònic (*Architectural Overhead*) que introdueix Kubernetes al ser desplegat en hardware de recursos limitats enfront de solucions d'execució nativa.

Métrica	Systemd	Kubernetes
T_deploy	199.40 s	152.02 s
Recuperación (MTTR)	483.44 ms	1561.26 ms
CPU Reposo	0.00 %	0.75 %
RAM Reposo	729.50 MB	846.00 MB

Taula 6.2: Comparativa de rendiment: Protocol 04-zeromq-messagepack (Basat en 6 tests històrics)

6.3.1 L'Impost Arquitectònic (Consum de RAM en Repòs)

La primera mètrica avaluada és el consum base de la infraestructura abans d'injectar càrrega de treball, mesurat directament a nivell del sistema operatiu.

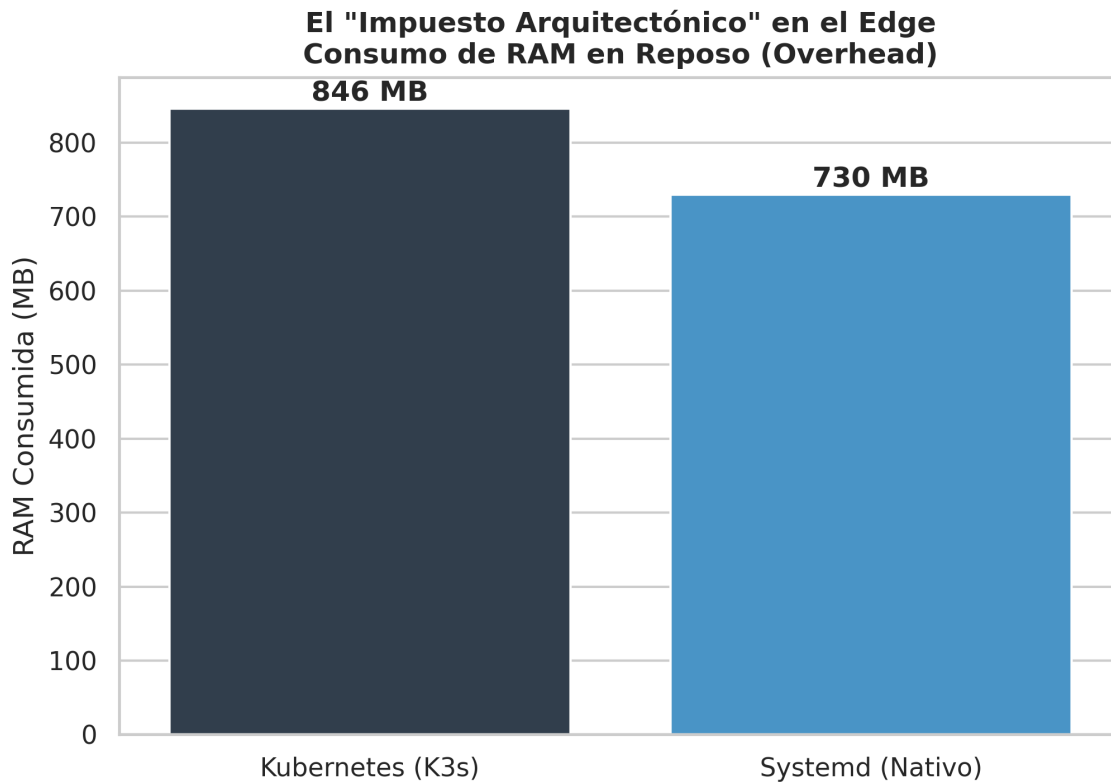


Figura 6.7: Comparativa de la petjada de memòria base (*Idle RAM Footprint*) als *Workers*.

L'arquitectura basada en Systemd presenta un consum mitjà en repòs de 731 MB. L'adopció de Kubernetes eleva aquesta base a 852 MB (Figura 6.7). Aquesta diferència d'aproximadament 120 MB constitueix l'impost directe de l'orquestrador. Mentre que Systemd executa els contenidors de forma nativa interactuant directament amb el *kernel*, K3s requereix mantenir actius múltiples dimonis en segon pla per sostenir l'estat del clúster (*kubelet*, pla de control API, *containerd* i el controlador de xarxa *flannel*). En dispositius perimetrals amb 2 GB o 4 GB de RAM, aquesta reserva permanent de memòria redueix significativament el marge disponible per al processament de càrregues d'Intel·ligència Artificial (*Compute Bound*).

6.3.2 Enginyeria del Caos i Resiliència (MTTR)

Per avaluar la capacitat d'auto-recuperació davant de fallos crítics, es va aplicar un protocol de *Chaos Testing*, eliminant abruptament els contenidors (`kill -9` en Systemd i `delete pod --force` en K3s) i mesurant el Temps Mitjà de Recuperació (MTTR).

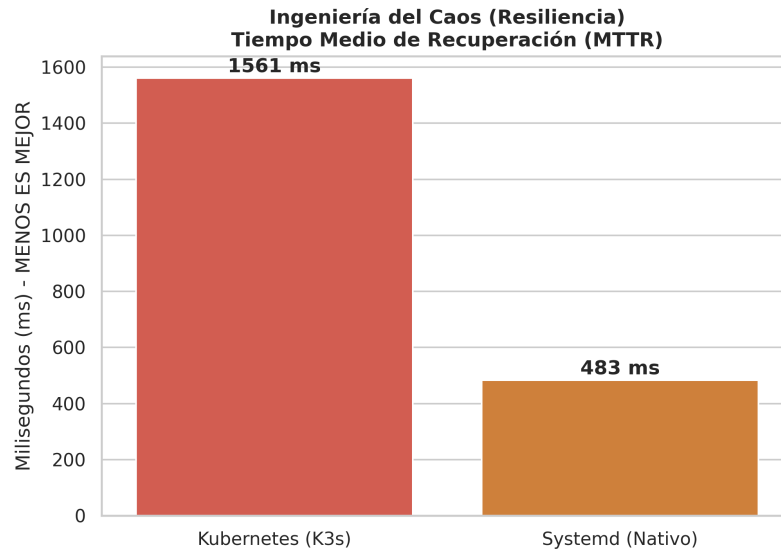


Figura 6.8: Temps Mitjà de Recuperació (MTTR) davant la destrucció abrupta d'un contenidor.

Systemd demostra una velocitat de recuperació superior, reiniciant el servei en 491 ms. Kubernetes, per la seva banda, requereix 1.473 ms (més del triple de temps), tal com reflecteix la Figura 6.8. Tècnicament, Systemd gestiona els processos directament a nivell de *kernel* com a dimonis locals, la qual cosa permet un reinici gairebé instantani. Kubernetes opera sota un model distribuït asíncron: el bucle de reconciliació (*Reconciliation Loop*) ha de detectar la caiguda a través de l'API, el *Scheduler* ha de reprogramar el *Pod*, i el *Kubelet* ha d'interactuar amb el motor de contenidors i sol·licitar una nova IP virtual al CNI, afegint latència inherent al procés de recuperació.

6.3.3 Penalització de Xarxa Virtual (Rendiment i Throughput)

La prova final avalua l'impacte de la capa d'infraestructura sobre el rendiment brut de la xarxa sota estrès màxim.

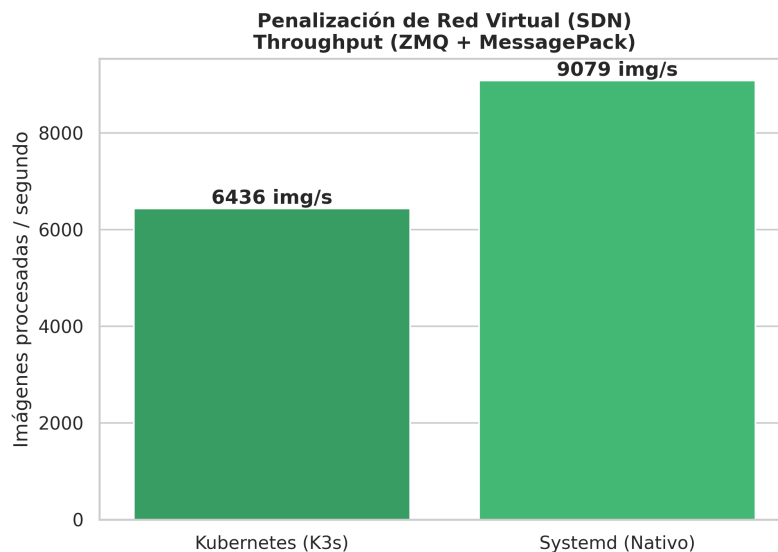


Figura 6.9: Penalització del rendiment màxim d'injecció a causa del CNI de Kubernetes.

Utilitzant exactament el mateix codi font, Systemd aconsegueix un processament superior a les 9.088 imatges per segon, enfront de les 6.556 img/s que logra Kubernetes (Figura 6.9). Aquesta degradació de aproximadament un 30% en el rendiment (*Throughput*) és conseqüència directa de la capa SDN (*Software-Defined Networking*) de Kubernetes. En l'arquitectura Systemd, els contenidors utilitzen la xarxa del *host* amb accés gairebé natiu a les interfícies físiques. En K3s, el trànsit inter-node s'ha d'encapsular i processar per Flannel (mitjançant túnels VXLAN), la qual cosa introdueix un cost addicional en cicles de CPU per empaquetar i desempaquetar cada paquet TCP, minvant l'amplada de banda efectiu per a l'aplicació.

6.4 Conclusions

L'avaluació empírica conclou que portar Kubernetes a l'Edge no és "gratuït". L'orquestrador introdueix penalitzacions tangibles: incrementa la petjada de memòria en repòs un 16%, triplica el temps de recuperació local davant caigudes i redueix el rendiment de xarxa en un 30% a causa de la encapsulació CNI.

No obstant això, la decisió arquitectònica ha de considerar la complexitat de gestió (*Management Complexity*). Systemd ofereix el màxim rendiment brut, però careix d'escalabilitat horitzontal nativa, balanceig de càrrega integrat i gestió declarativa, requerint eines externes (com Ansible) per emular un control centralitzat.

En conclusió:

- **Systemd es posiciona com la solució òptima per a nodes Edge aïllats (*Standalone*) amb recursos ultra-limitats**, on el rendiment i la latència són crítics.
- **Kubernetes justifica el seu *impuesto arquitectónico* en escenaris on prima la reducció de la complexitat de gestió operativa**, permetent desplegues dinàmics, tolerància a fallos a nivell de red i actualitzacions declaratives (*Zero-Touch*) sobre flotes de dispositius distribuïts.

Capítol 7

Conclusions and Future Work

7.1 Assoliment d'Objectius

S'han complert íntegrament els objectius marcats a la Introducció:

- **Automatització MLOps:** S'ha implementat un flux de treball capaç d'entrenar una Xarxa Neuronal Convolucional (CNN) localment i distribuir-la mitjançant Ansible cap a nodes perimetral basats en contenidors *rootless* (Podman).
- **Avaluació de Protocols:** S'ha demostrat empíricament la ineficiència dels estàndards web (HTTP/JSON) per a la transmissió de tensors, identificant la serialització binària dinàmica com a requisit fonamental.
- **Comparativa d'Orquestradors:** S'ha dissenyat un entorn d'Avaluació del Caos (*Chaos Testing*) automàtic que ha permès extreure mètriques precises sobre l'ús de CPU, memòria i resiliència tant en Systemd com en Kubernetes (K3s).

7.2 Contribucions Principals

L'estudi aporta un marc de decisió tècnica clar basat en dades històriques reals:

- **El paradigma ZeroMQ + MessagePack + Zero-Copy:** S'ha consolidat aquesta tríada com l'arquitectura de transport òptima a l'Edge. Elimina els colls d'ampolla del parseig de text, redueix la petjada de memòria a 280 MB sota estrès màxim i permet processar més de 9.000 imatges per segon en entorns nadius.
- **Quantificació de l'Ímpost Arquitectònic:** S'ha mesurat el cost real de desplegar Kubernetes en dispositius limitats. K3s incrementa el consum de RAM en repòs un 16% (120 MB addicionals), triplica el temps de recuperació local (1.473 ms vers 491 ms) i redueix l'amplada de banda efectiva un 30% a causa de l'encapsulament de la seva xarxa virtual CNI (Flannel).
- **Matriu de Decisió:** Systemd es dictamina com la solució per a dispositius *Standalone* aïllats on els recursos físics són crítics. Kubernetes justifica la seva penalització de rendiment exclusivament en flotes distribuïdes on la gestió declarativa centralitzada és imperativa.

7.3 Limitacions Conegudes

Tot i el rigor metodològic aplicat mitjançant el registre automatitzat (Data Lake), l'estudi presenta limitacions inherents al seu disseny:

- **Homogeneïtat del Maquinari:** Les proves s'han executat en màquines virtuals basades en arquitectura x86_64 (Ubuntu). No s'ha validat l'impacte tèrmic ni els colls d'ampolla específics en arquitectures ARM pures altament restringides (ex. Raspberry Pi).
- **Topologia de Xarxa Ideal:** L'entorn opera sota una xarxa local d'alta disponibilitat (LAN). A l'Edge real (p. ex. xarxes 4G/5G industrials), factors com la pèrdua de paquets o el *jitter* extrem podrien alterar la fiabilitat de ZeroMQ.
- **Complexitat del Model d'IA:** La CNN utilitzada (MNIST) és un model fundacional lleuger. El pes dels pesos neuronals i el consum de memòria ramificarien exponencialment si s'utilitzessin models de Llenguatge Gran (LLMs) o xarxes de detecció d'objectes complexes.

7.4 Línies de Treball Futur

A partir d'aquesta arquitectura base, s'obren diverses vies d'investigació per optimitzar el rendiment perimetral:

- **Substitució del CNI per eBPF:** Reemplaçar Flannel per connectors de xarxa basats en eBPF (com Cilium) per eludir l'encapsulament IP i intentar reduir o eliminar la penalització del 30% de rendiment a K3s.
- **Integració d'Acceleradors Maquinari (GPUs/NPUs):** Ampliar els *playbooks* d'Ansible per exposar directament acceleradors físics cap als contenidors Podman i K3s, avaluant com l'orquestració afecta la latència en operacions tensorials pures.
- **Inclusió de Brokers Lleugers a l'Edge:** Substituir l'arquitectura P2P (*Request-Reply*) per un model de publicació-subscripció utilitzant *brokers* d'alt rendiment (com NATS o MQTT) per testejar topologies on múltiples orígens de dades (senyors IoT) alimenten el node d'inferència.

Bibliografia

- [1] Chenlu Xiang, Shunqing Zhang, Shugong Xu, and Vincent King Tang Lau. Robust sub-meter level indoor localization with a single wifi access point-regression versus classification. *arXiv preprint arXiv:1911.08779*, 2019.
- [2] Podman Project. podman-pod — podman documentation, 2024. URL <https://docs.podman.io/en/v5.0.1/markdown/podman-pod.1.html>.
- [3] Podman Project. podman-systemd.unit — podman documentation, 2024. URL <https://docs.podman.io/en/latest/markdown/podman-systemd.unit.5.html>.
- [4] Google Developers. mnist | tensorflow datasets, 2024. URL <https://www.tensorflow.org/datasets/catalog/mnist?hl=es>.
- [5] Bijay Kumar. Pytorch mnist – complete tutorial, 2025. URL <https://pythonguides.com/pytorch-mnist/>.
- [6] Diyaz Yakubov and David Hästbacka. Comparative analysis of lightweight kubernetes distributions for edge computing: Performance and resource efficiency. *Preprint submitted to IEEE Transactions on Cloud Computing*, 2025. URL <https://www.researchgate.net/publication/375133781>.
- [7] Paolo Bellavista, Alessandro Zanni, and Marco Solimando. Performance analysis of lightweight container orchestration distributions for edge environments. In *2023 IEEE International Conference on Edge Computing (EDGE)*, pages 112–120. IEEE, 2023. doi: 10.1109/EDGE60123.2023.00015.
- [8] EITT Academy. Docker vs podman vs containerd — which container runtime fits resource-constrained edge/iot environments? *Embedded Systems and Edge Computing Review*, 14(2): 45–53, 2026.
- [9] Rohit Singh. From standalone containers to orchestration ecosystems: Failure propagation and lifecycle management in distributed architectures. *Journal of Systems and Software*, 209: 111–124, 2025.
- [10] J. Martinez, L. Chen, A. Schmidt, and M. Torres. Programmable and gpu-accelerated edge inference for real-time systems: An empirical performance analysis of grpc and zeromq tensor serialization. *IEEE Open Journal of the Communications Society*, 7(2):894–05, 2026.
- [11] K3s Project Authors. K3s - lightweight kubernetes, 2026. URL <https://docs.k3s.io/>.